



TEST SUITE DEVELOPMENT AND ANALYSIS FOR MAX CLIQUE ALGORITHMS

Summer 2025

Abstract

This document presents the implementation of graph theory concepts related to graph properties and clique-finding, along with a detailed overview of the functions defined, scope of testing, and key accomplishments achieved throughout the project.

Student: Clara Elliott | Supervisor: Jim Nastos
Okanagan College Kelowna

Table of Contents

Introduction	3
Background	3
What is a Maximum Clique?	3
Common Node Structures	3
Summary of Tech Stack.....	3
Functions Developed	4
Check Functions	4
Function: is_clique(G, v).....	4
Function: is_prime(G)	4
Function: is_module(G, subset)	4
Function: is_split(G)	4
Function: is_p4(G, v)	4
Function: is_c4(G, v).....	4
Function: is_quasi_threshold(G, certifying=None)	4
Function: is_quasi_threshold_2(G).....	5
Find Functions	5
Function: find_all_p4(G)	5
Function: find_all_c4(G).....	5
Function: find_all_p4_bfs(G)	5
Function: find_all_c4_bfs_broken(G_original)	5
Maximum Clique Algorithms	5
Algorithm: branch_and_bound(G).....	5
Algorithm Helper: greedy_coloring_heuristic(G, order)	6
Algorithm: Bron_Kerbosch(G).....	6
Algorithm: max_clique(G, nodes=None)	6
Algorithm: max_clique_with_steps(G, nodes=None)	6
Algorithm: custom_with_greedy(G, str_mode, nodes=None)	6
Algorithm: custom_with_greedy_steps(G, str_mode, nodes=None)	6
Algorithm: custom_with_partial_greedy(G, str_mode, nodes=None)	7
Algorithm: custom_with_partial_greedy_steps(G, str_mode, nodes=None)	7
Additional Functions	7
Function: read_dimacs_clq(source).....	7

Function: generate_trivially_perfect_graph(n, seed=None)	7
Disabled Functions	7
Function: bors_kerbosch_naive(R, P, X, G, C)	7
Function: bors_kerbosch_optimized(R, P, X, G, C)	7
Function: Bron_Kerbosch_Pivoting(G)	7
Function: testing_subgraph(G, v)	8
Function: make_neighbor_functor_nx(G)	8
Function: create_compliment(G)	8
Function: find_all_p4_bfs_old(G)	8
Function: find_cliques_with_steps(G, nodes=None)	8
Project Achievements	9
Test Files: Involved Functions	9
Test Files: Test and Assertion Count	13
Graph Properties: Filtering Results	15
Conclusion	16

Introduction

This report outlines the project work completed in support of James Nastos's work in Graph Theory, focusing on the development and analysis of algorithms used in maximum clique finding. The goal of this work was to assist in the creation and testing of functions that support graph property validation. Additionally, this work aimed to develop extensive test suites to confirm algorithm correctness and evaluating their performance by comparing runtimes.

Background

What is a Maximum Clique?

A **clique** in graph theory is a subset of nodes of an undirected graph where for every two distinct nodes in the subset there is an edge connecting them.

A clique is **maximal** if, and only if, it is not a proper subgraph of another clique. Meaning, no other distinct nodes in the graph are adjacent to all nodes already in the clique.

When a maximal clique is as large as or larger than the largest maximal clique found in the graph it is **maximum**.

Common Node Structures

A **complete graph (K_n)** is an entire graph where for every two distinct nodes there is an edge connecting them. A complete graph is, by definition, also a clique.

A **path graph (P_n)** is a graph consisting of two nodes with exactly degree-1 and every other node with exactly degree-2. A path graph can be drawn such that all nodes and edges lie on a single straight line.

A **cycle graph (C_n)** is a graph consisting of nodes each with exactly degree-2 that form a single closed loop.

A **bipartite graph ($K_{n,m}$)** is a graph where the nodes can be split into two independent sets such that no nodes in set 1 share an edge with any other node in set 1 and no nodes in set 2 share an edge with any other node in set 2.

Summary of Tech Stack

Python – The programming language chosen to define functions and test cases.

Unittest – The testing framework used for defining and running test cases.

NetworkX – A Python library that provides a comprehensive set of graph theory tools.

Matplotlib – A Python library widely used for data visualization, used here to visualize graphs.

Functions Developed

Check Functions

These functions check for specific graph properties and return a Boolean value. They are used throughout multiple functions that would otherwise be repeating the same code.

Function: `is_clique(G, v)`

This function takes in a NetworkX Graph object (G) and a list of nodes (v) and returns True if the nodes of v form a valid clique in G.

Function: `is_prime(G)`

This function takes in a NetworkX Graph object (G) and returns True if G is modularly prime.

This function uses `is_module(G, subset)` as a helper.

This function was created with the help of Artificial Intelligence (ChatGPT).

Function: `is_module(G, subset)`

This function takes in a NetworkX Graph object (G) and a set of nodes (subset) and returns True if the nodes of subset form a valid module in G.

This function was created with the help of Artificial Intelligence (ChatGPT).

Function: `is_split(G)`

This function takes in a NetworkX Graph object (G) and returns True if G and Complement-G are both chordal.

This function uses NetworkX function `is_chordal(G)` as a helper.

Function: `is_p4(G, v)`

This function takes in a NetworkX Graph object (G) and a list of four nodes (v) and returns True if the nodes of v form a valid 4-path in G.

Function: `is_c4(G, v)`

This function takes in a NetworkX Graph object (G) and a list of four nodes (v) and returns True if the nodes of v form a valid 4-cycle in G.

Function: `is_quasi_threshold(G, certifying=None)`

This function takes in a NetworkX Graph object (G), and optionally a Boolean (certifying), and returns True if G is quasi threshold.

This function uses `find_all_p4_bfs(G)` and `find_all_c4(G)` as helper functions.

This function is not optimal but is correct.

Function: `is_quasi_threshold_2(G)`

This function takes in a NetworkX Graph object (G) and returns True if G is quasi threshold.

Find Functions

These functions search for specific node patterns in a given graph and return a list of frozensets containing the nodes that match the pattern. Frozensets were used so testing would not rely on exact ordering.

Function: `find_all_p4(G)`

This function takes in a NetworkX Graph object (G), iterates through every combination of four nodes, and returns a list of frozensets containing valid 4-paths.

Function: `find_all_c4(G)`

This function takes in a NetworkX Graph object (G), iterates through every combination of four nodes, and returns a list of frozensets containing valid 4-cycles.

Function: `find_all_p4_bfs(G)`

This function takes in a NetworkX Graph object (G), traverses G using breadth first search logic, and returns a list of frozensets containing valid 4-paths.

Function: `find_all_c4_bfs_broken(G_original)`

This function takes in a NetworkX Graph object (G_original), makes a copy of G_original (G), traverses G using breadth first search logic, and returns a list of frozensets containing valid 4-cycles.

This function contains a critical failure when removing an edge from G, this action has the potential to create 4-cycles.

Maximum Clique Algorithms

These functions search for the maximum clique in a given graph and return the largest set of nodes from the graph that form a valid clique.

Algorithm: `branch_and_bound(G)`

Branch and bound is a form of optimized problem-solving that involves breaking down a problem into smaller sub-problems and used a bounding function to eliminate the sub-problems that cannot contain the optimal solution.

When applied to clique-finding, the bounding function is used to prune subtrees that cannot lead to a larger clique than what is already found.

This function returns a set containing the nodes in the largest clique found (best_clique).

This function uses `greedy_coloring_heuristic(G, order)` as a helper method.

This function was created with the help of Artificial Intelligence (ChatGPT).

Algorithm Helper: `greedy_coloring_heuristic(G, order)`

A greedy colouring heuristic is an algorithm that assigns a colour, represented by an integer, to each node in a graph such that no two adjacent nodes are assigned the same colour.

This function returns a dictionary ordering (coloring) of the nodes based on their colouring values.

This function was created with the help of Artificial Intelligence (ChatGPT).

Algorithm: `Bron_Kerbosch(G)`

The Bron Kerbosch algorithm is traditionally a recursive backtracking algorithm used by NetworkX's `find_cliques()` function to find all maximal cliques in a graph.

This version is an iterative algorithm using the same principles while avoiding the overhead of the recursive calls.

This function was created based off a post on Stack Overflow and edited with the help of Artificial Intelligence (ChatGPT).

<https://stackoverflow.com/questions/28406314/iterative-version-of-the-bron-kerbosch-algorithm>

Algorithm: `max_clique(G, nodes=None)`

This function is a version of the Bron Kerbosch iterative algorithm that uses bounding to prune subtrees that cannot lead to a larger clique than what is already found.

This function was created with the help of Artificial Intelligence (ChatGPT).

Algorithm: `max_clique_with_steps(G, nodes=None)`

This function is near-identical to `max_clique(G, nodes=None)`.

Difference: keeps a count of steps completed and returns along with the maximum clique.

Algorithm: `custom_with_greedy(G, str_mode, nodes=None)`

This function is a version of the `max_clique(G, nodes=None)` function that uses NetworkX's `greedy_color()` function as a bounding function to prune subtrees that cannot lead to a larger clique than what is already found.

The bounding function is used every iteration.

The parameter `str_mode` is used to specify the coloring method to be used in NetworkX's `greedy_color()` function.

This function was created with the help of Artificial Intelligence (ChatGPT).

Algorithm: `custom_with_greedy_steps(G, str_mode, nodes=None)`

This function is near-identical to `custom_with_greedy(G, str_mode, nodes=None)`.

Difference: keeps a count of steps completed and returns along with the maximum clique.

Algorithm: `custom_with_partial_greedy(G, str_mode, nodes=None)`

This function is a version of the `custom_with_greedy(G, str_mode, nodes=None)` function that uses NetworkX's `greedy_color()` function as a bounding function to prune subtrees that cannot lead to a larger clique than what is already found.

The bounding function is only used once every three iterations in an attempt to reduce computation overhead.

The parameter `str_mode` is used to specify the coloring method to be used in NetworkX's `greedy_color()` function.

This function was created with the help of Artificial Intelligence (ChatGPT).

Algorithm: `custom_with_partial_greedy_steps(G, str_mode, nodes=None)`

This function is near-identical to `custom_with_partial_greedy(G, str_mode, nodes=None)`.

Difference: keeps a count of steps completed and returns along with the maximum clique.

Additional Functions

Function: `read_dimacs_clq(source)`

This function takes a DIMACS.clq string file path (`source`) and returns a NetworkX Graph object.

Function: `generate_trivially_perfect_graph(n, seed=None)`

This function takes an integer (`n`) that defines the number of nodes and recursively builds a NetworkX Graph object.

This function returns a trivially perfect NetworkX Graph object.

Disabled Functions

Function: `bors_kerbosch_naive(R, P, X, G, C)`

This function is a simple, recursive implementation of the Bron Kerbosch (misspelt as bors kerbosch) algorithm.

This function was disabled due to a preference for an iterative method to save overhead.

Function: `bors_kerbosch_optimized(R, P, X, G, C)`

This function is near-identical to `bors_kerbosch_naive(R, P, X, G, C)`.

Difference: uses a bounding function to prune subtrees that cannot lead to a larger clique than what is already found.

This function was disabled due to a preference for an iterative method to save overhead.

Function: `Bron_Kerbosch_Pivoting(G)`

This function was later improved and renamed `max_clique(G)`.

Function: testing_subgraph(G, v)

This function was originally used as a visual aid when becoming familiar with the NetworkX draw_graph() function.

This function was disabled due to no longer fulfilling needs.

Function: make_neighbor_functon_nx(G)

This function returns a callable for listing out all neighbor nodes of the vertex provided.

This function was disabled due to no longer fulfilling needs.

Function: create_compliment(G)

This function uses NetworkX's built-in complement(G) function to create a complement of a given graph.

This function was disabled due to redundancy.

Function: find_all_p4_bfs_old(G)

This function takes in a NetworkX Graph object (G), traverses G using breadth first search logic, and returns a list of frozensets containing valid 4-paths.

This function was disabled due to logical errors and was re defined as find_all_p4_bfs(G).

Function: find_cliques_with_steps(G, nodes=None)

This function was an attempt to edit the source code for NetworkX's built-in function find_cliques(G) to count the steps the algorithm takes during runtime.

This function was disabled due to the edits required changing the fundamentals too drastically.

Project Achievements

Test Files: Involved Functions

Below is a series of tables showing all the test files written and which functions were involved in the process of testing regardless of if they were the focus of the test or not.

TEST FILE NAME	INVOLVED FUNCTIONS
test_is_c4.py	is_c4(G, v)
test_is_chordal.py	networkX is_chordal(G)
test_is_clique.py	is_clique(G, v)
test_is_module.py	is_module(G, subset)
test_is_p4.py	is_p4(G, v)
test_is_prime.py	is_prime(G) is_module(G, subset)
test_is_split.py	is_split(G) networkX is_chordal(G) is_clique(G, v)
test_is_quasi_threshold.py	is_quasi_threshold(G) find_all_p4(G) find_all_c4(G)
test_is_quasi_threshold_2.py	is_quasi_threshold_2(G) find_all_p4(G) find_all_c4(G)

TEST FILE NAME	INVOLVED FUNCTIONS
test_find_all_p4.py	is_p4(G, v) find_all_p4(G)
test_find_all_p4_bfs.py	is_p4(G, v) find_all_p4_bfs(G)
test_find_all_c4.py	is_c4(G, v) find_all_c4(G)
test_find_all_c4_bfs.py	is_c4(G, v) find_all_c4_bfs_broken(G)
test_find_all_p4_on_DIMACS.py	is_p4(G, v) find_all_p4(G) read_dimacs_clq(source)
test_find_all_p4_bfs_on_DIMACS.py	is_p4(G, v) find_all_p4_bfs(G) read_dimacs_clq(source)
test_find_all_c4_on_DIMACS.py	is_c4(G, v) find_all_c4(G) read_dimacs_clq(source)
test_find_all_c4_bfs_on_DIMACS.py	is_c4(G, v) find_all_c4_bfs_broken(G) read_dimacs_clq(source)

TEST FILE NAME	INVOLVED FUNCTIONS
test_branch_and_bound.py	is_clique(G, v) branch_and_bound(G)
test_bron_kerbosch_pivoting.py	is_clique(G, v) Bron_Kerbosch_Pivoting(G)
test_bron_kerbosch_pruning.py	is_clique(G, v) max_clique(G)
test_bron_kerbosch.py	is_clique(G, v) Bron_Kerbosch(G)
test_independent_set_colouring.py	is_clique(G, v) custom_with_greedy(G, str_mode)
test_independent_set_partial_greedy.py	is_clique(G, v) custom_with_partial_greedy(G, str_mode)
test_largest_first_colouring.py	is_clique(G, v) custom_with_greedy(G, str_mode)
test_largest_first_partial_greedy.py	is_clique(G, v) custom_with_partial_greedy(G, str_mode)
test_random_sequential_colouring.py	is_clique(G, v) custom_with_greedy(G, str_mode)
test_random_sequential_partial_greedy.py	is_clique(G, v) custom_with_partial_greedy(G, str_mode)

*This set of comparison tests is being run on 25-node subgraphs of DIMACS graphs to ensure correctness.

TEST FILE NAME	INVOLVED FUNCTIONS
test_branch_and_bound_against_built_in.py	is_clique(G, v) branch_and_bound(G)
test_bron_kerbosch_pruning_against_built_in.py	is_clique(G, v) max_clique(G)
test_bron_kerbosch_against_built_in.py	is_clique(G, v) Bron_Kerbosch(G)
test_independent_set_colouring_against_built_in.py	is_clique(G, v) custom_with_greedy(G, str_mode)
test_independent_set_partial_greedy_against_built_in.py	is_clique(G, v) custom_with_partial_greedy(G, str_mode)
test_largest_first_colouring_against_built_in.py	is_clique(G, v) custom_with_greedy(G, str_mode)
test_largest_first_partial_greedy_against_built_in.py	is_clique(G, v) custom_with_partial_greedy(G, str_mode)
test_random_sequential_colouring_against_built_in.py	is_clique(G, v) custom_with_greedy(G, str_mode)
test_random_sequential_partial_greedy_against_built_in.py	is_clique(G, v) custom_with_partial_greedy(G, str_mode)

*This set of comparison tests is being run on 50-node subgraphs of DIMACS graphs to ensure scalability.

TEST FILE NAME	INVOLVED FUNCTIONS
test_branch_and_bound_against_built_in.py	is_clique(G, v) branch_and_bound(G)
test_bron_kerbosch_pruning_against_built_in.py	is_clique(G, v) max_clique(G)
test_bron_kerbosch_against_built_in.py	is_clique(G, v) Bron_Kerbosch(G)
test_independent_set_colouring_against_built_in.py	is_clique(G, v) custom_with_greedy(G, str_mode)
test_independent_set_partial_greedy_against_built_in.py	is_clique(G, v) custom_with_partial_greedy(G, str_mode)
test_largest_first_colouring_against_built_in.py	is_clique(G, v) custom_with_greedy(G, str_mode)
test_largest_first_partial_greedy_against_built_in.py	is_clique(G, v) custom_with_partial_greedy(G, str_mode)
test_random_sequential_colouring_against_built_in.py	is_clique(G, v) custom_with_greedy(G, str_mode)
test_random_sequential_partial_greedy_against_built_in.py	is_clique(G, v) custom_with_partial_greedy(G, str_mode)

TEST FILE NAME	INVOLVED FUNCTIONS
test_execution_times_50v.py	is_clique(G, v) max_clique(G) custom_with_greedy(G, str_mode) custom_with_partial_greedy(G, str_mode) read_dimacs_clq(source)
test_execution_times_60v.py	is_clique(G, v) max_clique(G) custom_with_greedy(G, str_mode) custom_with_partial_greedy(G, str_mode) read_dimacs_clq(source)
test_execution_times_70v.py	is_clique(G, v) max_clique(G) custom_with_greedy(G, str_mode) custom_with_partial_greedy(G, str_mode) read_dimacs_clq(source)
test_execution_times_80v.py	is_clique(G, v) max_clique(G) custom_with_greedy(G, str_mode) custom_with_partial_greedy(G, str_mode) read_dimacs_clq(source)
test_execution_times_90v.py	is_clique(G, v) max_clique(G) custom_with_greedy(G, str_mode) custom_with_partial_greedy(G, str_mode) read_dimacs_clq(source)

TEST FILE NAME	INVOLVED FUNCTIONS
test_generate_qt_graph_1.py	generate_trivially_perfect_graph(n) is_quasi_threshold(G) find_all_c4(G) find_all_p4_bfs(G)
test_generate_qt_graph_2.py	generate_trivially_perfect_graph(n) is_quasi_threshold(G) find_all_c4(G) find_all_p4_bfs(G)
test_is_qt_2_remove_edge.py	is_quasi_threshold(G) is_quasi_threshold_2(G) find_all_c4(G) find_all_p4_bfs(G)
test_is_qt_2_up_to_100v.py	is_quasi_threshold(G) is_quasi_threshold_2(G) find_all_c4(G) find_all_p4_bfs(G)
greedy_strats_test.py	is_clique(G, v) max_clique(G) custom_with_greedy(G, str_mode) read_dimacs_clq(source)
rand_prune_test.py	is_clique(G, v) max_clique(G) custom_with_greedy(G, str_mode) read_dimacs_clq(source)
sparse_bi_test.py	is_clique(G, v) max_clique(G) custom_with_greedy(G, str_mode) read_dimacs_clq(source)
test_partial_pruning.py	is_clique(G, v) max_clique(G) custom_with_partial_greedy(G, str_mode) read_dimacs_clq(source)
test_divanc.py	divanc(G) edge_niche_centrality(G)
test_edge_niche_centrality.py	edge_niche_centrality(G)

Test Files: Test and Assertion Count

A total of 9,436 assertions were completed across 1,811 test.

TEST FILE NAME	TEST COUNT	ASSERT COUNT
test_is_c4.py	32	90
test_is_chordal.py	10	10
test_is_clique.py	34	91
test_is_module.py	29	96
test_is_p4.py	32	90
test_is_prime.py	36	97
test_is_split.py	39	97
is_quasi_threshold()	43	67
is_quasi_threshold_2()	43	67
	298	705

TEST FILE NAME	TEST COUNT	ASSERT COUNT
test_find_all_p4.py	30	149
test_find_all_p4_bfs.py	30	149
test_find_all_c4.py	30	149
test_find_all_c4_bfs.py	30	149
test_find_all_p4_on_DIMACS.py	37	148
test_find_all_p4_bfs_on_DIMACS.py	37	148
test_find_all_c4_on_DIMACS.py	37	148
test_find_all_c4_bfs_on_DIMACS.py	37	148
	268	1188

TEST FILE NAME	TEST COUNT	ASSERT COUNT
test_branch_and_bound.py	25	122
test_bron_kerbosch_pivoting.py	25	122
test_bron_kerbosch_pruning.py	25	122
test_bron_kerbosch.py	25	122
test_independent_set_colouring.py	25	122
test_independent_set_partial_greedy.py	25	122
test_largest_first_colouring.py	25	122
test_largest_first_partial_greedy.py	25	122
test_random_sequential_colouring.py	25	122
test_random_sequential_partial_greedy.py	25	122
	250	1220

*This set of comparison tests is being run on 25-node subgraphs of DIMACS graphs to ensure correctness.

TEST FILE NAME	TEST COUNT	ASSERT COUNT
test_branch_and_bound_against_built_in.py	37	222
test_bron_kerbosch_pruning_against_built_in.py	37	222
test_bron_kerbosch_against_built_in.py	37	222
test_independent_set_colouring_against_built_in.py	37	222
test_independent_set_partial_greedy_against_built_in.py	37	222
test_largest_first_colouring_against_built_in.py	37	222
test_largest_first_partial_greedy_against_built_in.py	37	222
test_random_sequential_colouring_against_built_in.py	37	222
test_random_sequential_partial_greedy_against_built_in.py	37	222
	333	1998

*This set of comparison tests is being run on 50-node subgraphs of DIMACS graphs to ensure scalability.

TEST FILE NAME	TEST COUNT	ASSERT COUNT
test_branch_and_bound_against_built_in.py	37	222
test_bron_kerbosch_pruning_against_built_in.py	37	222
test_bron_kerbosch_against_built_in.py	37	222
test_independent_set_colouring_against_built_in.py	37	222
test_independent_set_partial_greedy_against_built_in.py	37	222
test_largest_first_colouring_against_built_in.py	37	222
test_largest_first_partial_greedy_against_built_in.py	37	222
test_random_sequential_colouring_against_built_in.py	37	222
test_random_sequential_partial_greedy_against_built_in.py	37	222
	333	1998

TEST FILE NAME	TEST COUNT	ASSERT COUNT
test_execution_times_50v.py	37	407
test_execution_times_60v.py	37	407
test_execution_times_70v.py	37	407
test_execution_times_80v.py	37	407
test_execution_times_90v.py	37	407
	185	2035

TEST FILE NAME	TEST COUNT	ASSERT COUNT
test_generate_qt_graph_1.py	10	10
test_generate_qt_graph_2.py	10	10
test_is_qt_2_remove_edge.py	15	30
test_is_qt_2_up_to_100v.py	57	171
greedy_strats_test.py	2 – 100x loop 3 – 32x loop	2
rand_prune_test.py	6 – 100x loop 1 – 3x loop	2
sparse_bi_test.py	6 – 100x loop	2
test_partial_pruning.py	2 – 100x loop 3 – 32x loop	2
test_divanc.py	15	27
test_edge_niche_centrality.py	14	36
	144	292

Graph Properties: Filtering Results

Using all simple, connected graphs available on n -nodes ($6 \leq n \leq 10$) filtering was performed in the following sequence.

Graphs were first filtered for primness in respect to modular decomposition. Resulting graphs were then filtered to satisfy the degree constraints ($2 < \text{degree} < n-1$). A third filter retained only those graphs, along with their complements, that are P_5 -free – that is, both graph and its complement had to be P_5 -free. The final filter identified graphs from this subset that are non-split.

	PRIME	$2 < \text{DEGREE} < n-1$	P_5 & Co- P_5 FREE	NON-SPLIT
6-node	26	6	0	0
7-node	260	76	0	0
8-node	4,670	1990	1	0
9-node	145,870	84,040	9	0
10-node	8,110,356	5,749,698	142	22

The results from filtering have been submitted and approved on The On-Line Encyclopedia Of Integer Sequences (OEIS).

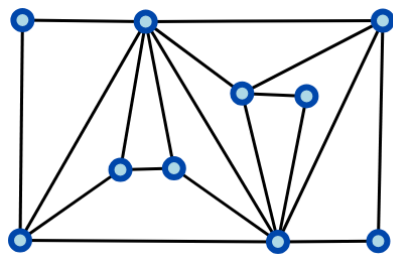
A385693 – Number of simple, undirected, prime graphs G on n unlabeled vertices such that neither G nor its complement contains a vertex of degree 1.

A385929 – Number of simple, undirected, prime graphs G on n unlabeled vertices such that neither G nor its complement contains a vertex of degree 1, and both G and its complement are P_5 -free.

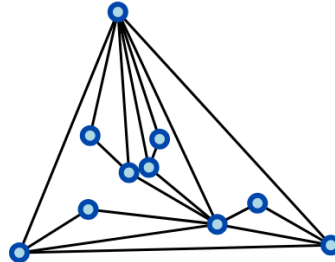
A385156 – Number of simple, undirected, prime, non-split graphs G on n unlabeled vertices such that neither G nor its complement contains a vertex of degree 1, and both G and its complement are P_5 -free.

The following graph have been submitted and approved on the House of Graphs.

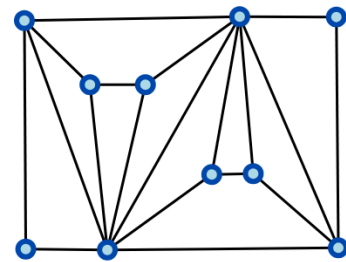
The following graphs are simple, undirected, prime graphs G on 10 unlabeled vertices with 19 edges such that neither G nor its complement contains a vertex of degree 1, and both G and its complement are P_5 -free.



Graph 53728



Graph 53753



Graph 53727

Conclusion

The goal of this work was to support the development and verification of functions related to various graph properties with a focus on creating a robust series of tests for maximum clique-finding algorithms. Through close collaboration with Jim Nastos and the use of ChatGPT for AI-assisted function generation and editing, a total of 31 functions were defined and tested across 1,811 tests and 9,436 assertions. While this fell short of the initial goal of 10,000 assertions, test coverage remains comprehensive in addressing both edge cases and scalability cases.

Most of the testing involved smaller graphs to ensure correctness and benchmark early-stage performance comparisons. Interestingly, performance tests using 50-90 node DIMACS subgraphs revealed that the simpler implementations, NetworkX's `find_cliques()` and the basic `max_clique()` functions, were consistently outperforming the functions designed to be optimized. It has been hypothesized that the advantages the optimizations provide will likely become more evident when applied to larger scale graphs, a key area for future testing.

In addition to optimizing clique finding, this work explored filtering simple connected graphs on n vertices, where $6 \leq n \leq 10$, to find those graphs that satisfy a set of strict structural constraints. These constraints included filtering for prime with respect to modular decomposition, no nodes of degree-1 in both the graph and its complement, P_5 free in both the graph and its complement, and having a non-split structure. Results from these tests were submitted, and accepted, to the House of Graphs and the Online Encyclopedia of Integer Sequences.

Overall, this work lays a solid foundation in benchmarking maximum clique-finding performance and offers insight into meaningful next steps for testing.